

ID	Ubicación	Síntoma observado	Principio comprometido	Impacto en el negocio	Severidad / Origen
H-01	Bib_Hacienda/Clases/Hacienda.cs Lineas (61-91-128-143-171-220-268-302-336-393-451)	La clase concentra actividades que pertenecen a áreas diferentes del sistema: administra inventario de vacunas, ventas, potreros, alimentación y aplicación de vacunas. Esto genera múltiples razones independientes para modificar la misma clase.	SRP	Riesgo alto: un cambio en cualquiera de estos procesos puede afectar funcionalidades no relacionadas y aumentar el tiempo de pruebas después de cada modificación.	Alta / Propio — PDF “Análisis Gloria Yuliana Peña Ibarguen y Análisis Juan Sebastián Jaramillo García”
H-02	Bib_Hacienda/Clases/Hacienda.cs Líneas (341-344-398-401-476-480-487-492-497-531-535)	En el método aplicar_vacuna() se identifica explícitamente el tipo concreto del animal mediante comprobaciones como res is Ternero y res is Novillo para decidir qué comportamiento ejecutar. Además, existen valores y límites de negocio escritos directamente en la lógica, como la comprobación cantidad > 100 durante la creación de lotes de vacunas, por lo que tanto la incorporación de nuevos tipos de res como los cambios en las reglas de negocio requieren modificar el código fuente existente.	OCP	los cambios en reglas de vacunación o la incorporación de nuevas categorías de res requieren modificar y volver a probar lógica existente, aumentando el tiempo y costo de mantenimiento y el riesgo de introducir regresiones en funcionalidades que ya operan correctamente.	Alta / Asistido

H-03	Bib_Hacienda/Clases/Hacienda.cs Lineas(43-44-45-46-77-156-288-322-372-429)	Hacienda construye directamente dependencias mediante expresiones como new Potrero (), new Venta (), new Bacteriana () y new Viva (), por lo que depende de implementaciones concretas.	DIP	Costo y riesgo medios: el acoplamiento dificulta sustituir componentes y aislar Hacienda en pruebas; cambios en esas implementaciones pueden propagarse a esta clase.	Media / Asistido
H-04	Bib_Hacienda/Clases/Potrero.cs Lineas (96-100-104)	La creación de animales se realiza utilizando directamente tipos concretos de reses, haciendo que Potrero conozca los detalles específicos de su construcción en vez de apoyarse en una abstracción.	DIP	Costo alto: cambiar la forma de crear reses o incorporar otras implementaciones obliga a modificar Potrero, aumentando el tiempo de cambio y el riesgo de regresiones.	Alta / Asistido
H-05	Bib_Hacienda/Clases/Potrero.cs Lineas(66-93)	La lógica de la clase está construida alrededor de únicamente tres tipos concretos de res y contiene decisiones específicas para ellos. Extender el dominio exige modificar la implementación existente.	OCP	Riesgo alto: agregar otro tipo de res exige alterar condiciones existentes, aumentando el tiempo de pruebas y la posibilidad de regresiones.	Alta / Asistido
H-06	Bib_Hacienda/Clases/Potrero.cs Lineas (41-115-121-127-133-140-141-142-143-169)	La clase combina coordinación de eventos, administración o incorporación de reses y validaciones	SRP	Costo alto: cambios en reglas de edad, creación de animales o eventos afectan la misma clase,	Alta / Propio — PDF “Análisis Isabella Gutiérrez Montoya”

		relacionadas con la edad de los animales. Estas funciones corresponden a responsabilidades diferentes.		aumentando el tiempo de mantenimiento y el riesgo de regresiones.	
H-07	Bib_Hacienda/Clases/Autenticacion.cs Lineas (29-69-77-104)	En una misma clase se encuentran responsabilidades relacionadas con la gestión de usuarios, la autenticación y la autorización. Son comportamientos que pueden evolucionar por motivos diferentes.	SRP	Tiempo y riesgo altos: modificar autenticación o permisos obliga a intervenir una clase con otras funciones de seguridad, ampliando las pruebas y el riesgo de regresiones.	Alta / Asistido
H-08	Bib_Hacienda/Clases/Autenticacion.cs Linea (123-128-133)	La asignación de permisos depende explícitamente de los roles actualmente conocidos. Cuando aparece un rol nuevo es necesario modificar el método de autorización existente.	OCP	Costo y riesgo altos: cada nuevo rol obliga a modificar autorización ya probada, aumentando mantenimiento y posibilidad de alterar permisos existentes.	Alta / Asistido
H-09	Bib_Hacienda/Clases/Novillo.cs Linea 28	Novillo redefine el manejo de la edad imponiendo un comportamiento particular respecto del definido por su jerarquía, lo que puede alterar las expectativas que tiene el código cliente sobre una res.	LSP	Riesgo medio: determinadas asignaciones de edad pueden producir resultados distintos o errores en ejecución, aumentando el tiempo de diagnóstico y corrección.	Media / Asistido
H-10	Bib_Hacienda/Clases/Ternero.cs Linea 23	La clase establece su propia lógica para asignar la edad, introduciendo	LSP	Riesgo medio: el error puede aparecer solo al usar Ternero	Media / Asistido

		condiciones particulares que pueden diferir del contrato esperado para el tipo base.		polimórficamente, aumentando el tiempo de diagnóstico y corrección.	
H-11	Bib_Hacienda/Clases/Cebon.cs Linea 23	Cebon introduce reglas propias para modificar la edad que pueden ser más restrictivas o diferentes de las establecidas por el contrato de la clase padre.	LSP	Riesgo medio: sustituir la clase base por Cebon puede provocar comportamientos inesperados, aumentando el tiempo de diagnóstico, corrección y pruebas.	Media / Asistido
H-12	Bib_Hacienda/Interfaces/IValidarInformacion.cs Lineas(15-18-21-24)	Interfaz que obliga a clientes a depender de métodos que no usan (validar Res, Potrero, Vacuna, Venta en un único contrato)	ISP	Costo medio: penaliza reutilización y testing, dificulta crear validadores específicos y aumenta el tiempo requerido para cambiar una validación.	Media / Asistido
H-13	P_mvchacienda/Servicios/PersistenciaService.cs Lineas(16-23, 41, 62, 95, 133, 175, 218, 275, 304, 346, 389, 455, 530, 602)	PersistenciaService mezcla validación/proxy con serialización a archivos y acceso a HttpContext.	SRP	Costo alto: cambiar persistencia o validación exige tocar la misma clase, aumentando el tiempo de mantenimiento y el riesgo de regresiones.	Alta / Propio — PDF “Análisis Juan Sebastián Jaramillo García”
H-14	P_mvchacienda/Servicios/PersistenciaService.cs — CargarVentas() 434 - 440 y carga de vacunas aprox. 492 -515 / 570 - 587	La reconstrucción de entidades depende de decisiones explícitas sobre tipos concretos (Ternero, Cebon,	OCP	Incorporar nuevos tipos de res o vacuna requiere modificar código existente de persistencia, elevando el	Alta / Asistido

		Novillo, Bacteriana, Viva).		costo y riesgo de futuras extensiones.	
H-15	P_mvchacienda/Servicios/PersistenciaService.cs — InicializarProxies(). 48 - 55 y acceso directo a archivos en métodos de guardar/cargar	PersistenciaService crea directamente infraestructura y validadores concretos y está acoplado al almacenamiento mediante archivos planos.	DIP	Costo alto: cambiar almacenamiento o validación requiere modificar esta clase, aumentando el tiempo de pruebas y el riesgo de regresiones.	Alta / Propio — PDF “Análisis Thomas Osorio Passos”
H-16	P_mvchacienda/Servicios/PotreroService.cs Líneas: (20-52-71-111)	PotreroService mezcla orquestación de dominio, validación/localización de reglas y persistencia con métodos de consulta/estadísticas en una sola clase	SRP	Cambio en creación/agregado de potreros o en almacenamiento /consulta obliga a tocar la misma clase; aumenta el riesgo de regresiones y el coste de mantenimiento.	Alta / Asistido
H-16	P_mvchacienda/Servicios/PotreroService.cs líneas 8-15	PotreroService depende directamente de las clases concretas Hacienda y PersistenciaService .	DIP	Costo alto: cambiar dominio o persistencia obliga a afectar este servicio, aumentando el tiempo de pruebas y el riesgo de regresiones.	Alta / Asistido
H-17	P_mvchacienda/Servicios/ResService.cs Líneas 8-15	ResService depende directamente de las clases concretas Hacienda y PersistenciaService ; además, PersistenciaService se inyecta, pero no	DIP	Costo alto: cambios en Hacienda o PersistenciaService obligan a modificar ResService; el acoplamiento y la dependencia no usada	Alta / Asistido

		se utiliza en la clase.		aumentan tiempo de pruebas y mantenimiento.	
H-18	P_mvchacienda/Servicios/ResService.cs Lineas 65-69	ResService determina las estadísticas mediante comprobaciones explícitas de tipos concretos (Ternero, Cebon, Novillo).	OCP	Tiempo y riesgo altos: un nuevo tipo de Res obliga a modificar ObtenerEstadísticas (), ampliar pruebas y asumir riesgo de regresiones.	Alta / Asistido
H-19	P_mvchacienda/Servicios/UsuarioService.cs líneas 10–16	UsuarioService depende directamente de la clase concreta PersistenciaService .	DIP	Un cambio en el mecanismo de persistencia obliga a afectar o reconfigurar directamente el servicio de usuarios. Dificulta sustituir archivos por BD, mocks u otra implementación y aumenta el costo de pruebas y mantenimiento.	Alta / Asistido
H-20	P_mvchacienda/Servicios/UsuarioService.cs líneas 18–99	UsuarioService concentra gestión de usuarios, validación de creación, autenticación, consultas, estadísticas y generación de Claims.	SRP	Tiempo y riesgo medios: cambios en autenticación, creación, consultas o seguridad obligan a modificar la misma clase y ampliar las pruebas.	Media / Asistido
H-21	P_mvchacienda/Servicios/VacunaService.cs líneas 8-15	VacunaService depende directamente de las clases concretas	DIP	Costo alto: cambiar dominio o persistencia obliga a afectar	Alta / Asistido

		Hacienda y PersistenciaService .		el servicio, aumentando tiempo de pruebas y riesgo de regresiones.	
H-22	P_mvchacienda/Servicios/VacunaService.cs líneas (24–36,132–137)	El servicio conoce explícitamente los tipos Bacteriana y Viva y decide su comportamiento según parámetros y comprobaciones de tipo.	OCP	Incorporar un nuevo tipo de vacuna obliga a modificar VacunaService en creación y estadísticas, aumentando costo y riesgo de extensión.	Alta / Propio — PDF “Análisis Thomas Osorio Passos”
H-23	P_mvchacienda/Servicios/VacunaService.cs líneas 50–91	VacunaService coordina lógica de dominio, carga de datos, modificación de inventario, persistencia de varias colecciones y composición de mensajes.	SRP	Tiempo y riesgo altos: cambios en aplicación de vacunas, persistencia, catálogo o mensajes afectan la misma clase y amplían las pruebas necesarias.	Alta / Asistido
H-24	P_mvchacienda/Servicios/VentaService.cs líneas 8–14	VentaService depende directamente de las clases concretas Hacienda y PersistenciaService además, PersistenciaService se inyecta, pero no se utiliza en la clase.	DIP	Costo medio: el acoplamiento y la dependencia innecesaria dificultan sustitución, pruebas y mantenimiento, aumentando el tiempo de cambio.	Media / Asistido
H-25	Bib_Hacienda/Eventos/PublisherPesoMin.cs Linea 25-27	El publicador decide el peso mínimo con comprobaciones de tipo concreto (res es Ternero, Cebon, Novillo).	OCP	Costo y tiempo altos: nuevos tipos de res requieren modificar y volver a probar este publicador, aumentando el	Alta / Asistido

				riesgo de regresiones.	
H-26	Bib_Hacienda /Eventos/ PublisherPesoVenta.cs Linea 27-29	El publicador decide el peso de venta con condiciones de tipo concreto.	OCP	Costo y tiempo altos: nuevos tipos de res obligan a modificar y probar nuevamente la lógica de eventos, con riesgo de regresiones.	Alta / Asistido
H-27	Bib_Hacienda /Eventos/ PublisherVacunacionCompletada.cs Linea 30-41	La clase verifica esquemas de vacunación usando res es Ternero, Novillo, Cebon.	OCP	Costo y tiempo altos: cada nueva categoría de res exige modificar y volver a probar la lógica de vacunación, aumentando el riesgo de regresiones.	Alta / Asistido
H-28	Bib_Hacienda/Clases/Validaciones/ ValidarPotrero.cs, ValidarRes.cs, ValidarVacuna.cs y ValidarVenta.cs Líneas 10–33	Cada validador concreto hereda de Validacion y sobrescribe las cuatro operaciones del contrato, pero solo implementa una; las otras tres lanzan NotImplementedException. Por ello, un ValidadorRes, ValidadorPotrero, ValidadorVacuna o ValidadorVenta no puede sustituir de forma segura a Validacion para todas las operaciones que el tipo base promete.	LSP	Riesgo alto: código que trabaje polimórficamente con Validacion puede fallar únicamente en ejecución según el subtipo recibido. Esto aumenta el costo de pruebas, obliga a conocer qué método admite cada implementación y eleva el riesgo de interrupciones al reutilizar o extender el mecanismo de validación.	Alta / Propio — PDF “Análisis Isabella Gutiérrez Montoya”

H-29	p_mvchacienda/Controllers/VacunaController.cs Líneas 56–116, 94–116	El controlador decide explícitamente entre los tipos de vacuna mediante tipoVacuna == "Bacteriana" y un else asumido como Viva, además de exigir parámetros distintos para cada caso. La capa de presentación conoce las variantes concretas y debe modificarse cuando el dominio incorpore otro tipo de vacuna.	OCP	Costo y tiempo altos: agregar una nueva clase de vacuna exige modificar también el controlador y volver a probar el flujo de creación web. El else que trata cualquier caso no bacteriano como Viva añade riesgo de clasificar incorrectamente nuevos tipos o entradas no previstas.	Alta / Propio — PDF “Análisis Thomas Osorio Passos”
H-30	p_mvchacienda/Controllers/PotreroController.cs Líneas 11–20 y 75–79	PotreroController mantiene referencias directas a las clases concretas Hacienda y PersistenciaService. Además, después de delegar la creación del potrero en PotreroService —que persiste internamente— el controlador vuelve a invocar directamente PersistenciaService para guardar los potreros. La capa de presentación queda así acoplada tanto al dominio concreto como al detalle de persistencia.	DIP	Costo y riesgo altos: cambiar el mecanismo de persistencia o la implementación concreta del dominio puede obligar a modificar el controlador. La escritura adicional incrementa también el tiempo de ejecución y el riesgo de operaciones redundantes o inconsistentes, además de dificultar pruebas aisladas.	Alta / Asistido
H-31	p_mvchacienda/Controllers/ResController.cs	ResController mantiene referencias directas	DIP	Costo y riesgo altos: cambios en Hacienda o	Alta / Asistido

	Líneas 11–22, 43–44, 113–116 y 165–169	a Hacienda y PersistenciaService y, en varias acciones, evita apoyarse únicamente en los servicios de aplicación: consulta directamente Hacienda, ejecuta operaciones de alimentación y venta sobre ella y después invoca PersistenciaService para guardar los cambios. La presentación depende de implementaciones concretas del dominio y de infraestructura.		en el mecanismo de persistencia pueden propagarse hasta la capa de presentación. Esto aumenta el tiempo de cambio, dificulta sustituir dependencias por dobles de prueba y eleva el riesgo de regresiones al modificar alimentación, venta o almacenamiento .	
--	--	---	--	---	--

Sugerencia de la herramienta refutada

La siguiente observación fue generada durante la revisión automatizada, pero se descarta después de contrastarla con el código.

Sugerencia: La clase Venta viola el principio de responsabilidad única (SRP) porque contiene información del potrero, la res, la fecha y el monto; por ello tendría varias responsabilidades.

Resultado: Refutada. No constituye una violación de SRP.

Evidencia en el código: En Bib_Hacienda/Clases/Venta.cs, líneas 9-28, la clase únicamente representa una venta. Sus cuatro campos —Potrero, Res, Fecha y Monto— se inicializan en el constructor y se exponen mediante propiedades. La clase no persiste datos, no calcula precios, no valida reglas, no genera mensajes y no coordina la interfaz de usuario.

Argumento técnico: SRP no se determina contando atributos. Una responsabilidad es una razón coherente de cambio. Los cuatro datos pertenecen al mismo concepto del dominio y cambian por una sola razón: una modificación en la forma de representar una venta. Por tanto, la clase mantiene alta cohesión y una única responsabilidad: conservar el estado de una venta de res.

Conclusión: La herramienta confundió cantidad de datos con cantidad de responsabilidades. El señalamiento es un falso positivo y no se incorpora a la matriz

como un nuevo hallazgo. Venta solo violaría SRP si además asumiera razones independientes de cambio, por ejemplo persistencia, facturación, notificación o presentación.

Trazabilidad de los hallazgos propios

Se conservan como propios únicamente los hallazgos cuya observación inicial puede verificarse en los análisis de lectura en frío. La formulación SOLID se precisó después, pero el problema y su ubicación general ya habían sido identificados por el equipo.

Hallazgo	Documento de origen	Relación con la observación inicial
H-01	Análisis Gloria Yuliana Peña Ibarguen; Análisis Juan Sebastián Jaramillo García	Ambos análisis identificaron que Hacienda concentraba la administración de potreros, ventas, vacunas y reglas de negocio, con alto riesgo y costo de cambio.
H-06	Análisis Isabella Gutiérrez Montoya	El análisis señaló que Potrero realizaba demasiadas tareas y reunía preocupaciones de capacidad, edad, incorporación y búsqueda de reses.
H-13	Análisis Juan Sebastián Jaramillo García	El análisis ubicó PersistenciaService.cs como una clase con múltiples responsabilidades y anticipó el alto impacto de cambiar el almacenamiento.
H-15	Análisis Thomas Osorio Passos	El análisis detectó el acoplamiento al almacenamiento en archivos de texto y el costo considerable de sustituirlos por una base de datos.
H-22	Análisis Thomas Osorio Passos	El análisis advirtió que incorporar otro tipo de vacuna obligaba a modificar el servicio, las reglas, los formularios y las vistas existentes.
H-28	Análisis Isabella Gutiérrez Montoya	El análisis identificó excepciones no implementadas en las clases hijas y el riesgo de quebrar la sustitución esperada en la jerarquía.
H-29	Análisis Thomas Osorio Passos	La observación inicial sobre los múltiples cambios necesarios al agregar una vacuna incluía los formularios y las vistas; la revisión posterior la localizó en VacunaController.

Tres puntos de dolor del negocio priorizados

La priorización parte de la situación descrita por el jefe: el sistema es crítico y funciona, pero un cambio pequeño tarda semanas, obliga a intervenir varios archivos y puede romper otra funcionalidad. Por tanto, cada punto se expresa como un dolor para el negocio; el principio SOLID se conserva únicamente como evidencia de la causa estructural. Los tres problemas seleccionados reúnen 27 de los 32 registros de la matriz (84,4 %).

Criterio principal: recurrencia: Se contó cuántas veces aparece cada causa en los 32 registros de la matriz: OCP = 10, DIP = 10, SRP = 7, LSP = 4 e ISP = 1. Por ello, OCP, DIP y SRP originan los tres puntos de dolor priorizados.

Primer criterio de desempate: severidad: Cuando dos causas tienen la misma frecuencia, se ubica primero la que concentra más hallazgos de severidad alta. OCP reúne 10 hallazgos altos; DIP reúne ocho altos y dos medios.

Segundo criterio de desempate: relación directa con el dolor empresarial: Se prioriza la causa que incide más directamente en el tiempo, el costo y el riesgo de atender nuevas solicitudes, así como en la capacidad de evolución requerida para los próximos tres años.

Regla de conteo: Se contaron registros de la matriz, no solamente identificadores únicos. H-16 cuenta dos veces porque documenta dos problemas distintos: uno relacionado con SRP y otro con DIP.

Orden	Punto de dolor del negocio	Causa	Recurrencia	Severidad
1	Agregar o modificar funcionalidades exige reabrir código ya probado	OCP	10 registros	10 altas
2	Un cambio local se propaga a varias áreas y pone en riesgo la continuidad operativa	DIP	10 registros	8 altas / 2 medias
3	Cada modificación obliga a revisar procesos no relacionados	SRP	7 registros	6 altas / 1 media

1. Agregar o modificar funcionalidades exige reabrir código ya probado

Hallazgos relacionados: H-02, H-05, H-08, H-14, H-18, H-22, H-25, H-26, H-27 y H-29. Son 10 de los 32 registros (31,3 %) y todos tienen severidad alta.

Punto de dolor: Incorporar una nueva categoría de res, un tipo de vacuna, un rol o una variación de una regla obligaba a modificar decisiones distribuidas en diferentes partes del sistema. Por eso una solicitud funcional pequeña podía exigir intervenir varios archivos, ampliar de manera importante las pruebas de regresión y asumir el riesgo de alterar reglas que ya funcionaban. Para el negocio, esto se traduce en entregas lentas,

mayor costo por cambio y pérdida de capacidad para responder oportunamente a nuevas necesidades.

Cómo se corrigió: La estructura se reorganizó para que las variaciones previsibles del negocio puedan incorporarse en puntos de extensión delimitados, sin reescribir los flujos ya probados. Las reglas, mensajes, cálculos y resultados existentes se conservaron; el cambio consistió en localizar cada variación en la parte que le corresponde. Así, una nueva categoría o regla puede añadirse con una intervención más localizada y con una regresión menor sobre el resto del sistema.

Justificación de la posición: Empata con el punto 2 en recurrencia, con 10 registros, pero ocupa el primer lugar por dos razones: sus 10 hallazgos son de severidad alta y expresa de forma directa el problema planteado por el jefe: el negocio solicita cambios pequeños, pero incorporarlos toma demasiado tiempo y obliga a tocar código existente. También es el dolor más alineado con el objetivo de que el sistema pueda evolucionar durante los próximos tres años.

2. Un cambio local se propaga a varias áreas y pone en riesgo la continuidad operativa

Hallazgos relacionados: H-03, H-04, H-15, H-16 (registro DIP), H-17, H-19, H-21, H-24, H-30 y H-31 (ResController, desarrollado como H-32 en el documento de justificaciones). Son 10 de los 32 registros (31,3 %): ocho de severidad alta y dos de severidad media.

Punto de dolor: Las áreas de negocio, presentación y almacenamiento estaban conectadas de manera tan directa que modificar una de ellas podía obligar a ajustar o volver a probar varias más. Esto dificulta aislar el efecto real de un cambio y aumenta la posibilidad de que una mejora en un proceso rompa otro que no debía verse afectado. En un sistema crítico, el resultado empresarial es mayor costo de análisis y pruebas, menor previsibilidad de los plazos y riesgo de detener la operación.

Cómo se corrigió: Se redujeron las dependencias directas entre las distintas áreas y se definieron límites claros sobre lo que cada proceso necesita de los demás. Las conexiones necesarias quedaron controladas en un único punto y se retiraron dependencias que no aportaban al comportamiento. Esto permite cambiar o probar una parte con menor arrastre sobre las otras, conservando la misma conducta observable.

Justificación de la posición: Tiene la misma recurrencia que el punto 1, pero queda en segundo lugar porque su efecto principal es amplificar el costo y el riesgo de los cambios, mientras el punto 1 representa directamente la incapacidad de incorporar lo que el negocio solicita. Aun así, precede al punto 3 porque aparece en más hallazgos, atraviesa más fronteras del sistema y compromete de manera directa la continuidad de una aplicación crítica.

3. Cada modificación obliga a revisar procesos no relacionados

Hallazgos relacionados: H-01, H-06, H-07, H-13, H-16 (registro SRP), H-20 y H-23. Son siete de los 32 registros (21,9 %): seis de severidad alta y uno de severidad media.

Punto de dolor: Algunos componentes atendían al mismo tiempo procesos de negocio distintos, como usuarios, ventas, potreros, vacunación, persistencia o mensajes. Por esa concentración, una modificación específica obligaba a comprender y volver a comprobar funciones que no estaban relacionadas con la solicitud. Para el negocio, esto significa más horas de diagnóstico y pruebas, mayor dependencia del conocimiento individual de quien mantiene el sistema y mayor probabilidad de regresiones inesperadas.

Cómo se corrigió: Las responsabilidades se redistribuyeron según procesos y razones de cambio del negocio. Cada parte quedó enfocada en una función más coherente, y la coordinación necesaria se mantuvo sin alterar las reglas ni las salidas del sistema. Con ello, los cambios futuros pueden concentrarse en un alcance menor y las pruebas pueden enfocarse en el proceso realmente afectado.

Justificación de la posición: Es el tercer problema más recurrente, con siete registros, frente a 10 en cada uno de los dos primeros. Su impacto en tiempo, costo y riesgo sigue siendo alto, pero aparece con menor frecuencia y su alcance está más concentrado. Por eso se prioriza después de la dificultad para extender el sistema y de la propagación de cambios entre áreas.

Comparación clave entre 1 y 2: ambos aparecen 10 veces. El número 1 queda antes porque sus 10 hallazgos son altos, mientras el número 2 tiene ocho altos y dos medios. Además, el primero representa directamente la demora para incorporar nuevas necesidades del negocio; el segundo explica principalmente por qué esos cambios se propagan y se vuelven riesgosos. En síntesis: primero se prioriza la capacidad de evolucionar y, enseguida, el control del riesgo con que se realiza esa evolución.

Hallazgos no incluidos en los tres primeros puntos: los cuatro registros LSP y el registro ISP continúan siendo válidos y fueron corregidos. No se descartan; simplemente quedan fuera de los tres dolores prioritarios porque son menos recurrentes.